

A1 · Speaking the Application Layer

CS425 Computer Networks · in class activity · everything runs on onyx.boisestate.edu

TURN THIS IN ON PAPER

before you leave the room
graded pass/fail

Names _____ Date _____

ROUND 1 · GET ON THE BOX, AND FIND YOUR INSTRUMENTS

One machine, everyone on it. If your answer differs from your neighbor's, one of you typed something wrong, because there is no second network to blame.

```
ssh <username>@onyx.boisestate.edu
cat /etc/redhat-release      # which Red Hat
uname -srm                  # kernel and architecture
who | wc -l                  # how many of us are on here right now
```

QUESTION	WHAT IT PRINTED
Red Hat release	
Kernel and architecture	
People logged in right now	

Now find out **which** version of each instrument you have, because that decides which switches work. There are at least three unrelated programs called **nc** and they take different flags; knowing which one is in front of you is the difference between a five second answer and a lost afternoon.

```
dig -v          curl --version      nc --version    ping -V
```

TOOL	VERSION, AND ANYTHING SURPRISING
dig	
curl	
nc	
ping	

ROUND 2 · dig: THE DOMAIN NAME SYSTEM

DNS is an application layer protocol. A real query goes to a real server and a real answer comes back, and all of it can be inspected.

```
dig boisestate.edu
```

FIND THIS IN THE OUTPUT	WRITE IT HERE
The status: field	
The QUESTION section	
ANSWER: type, TTL, address	
Query time:	

Now watch the TTL

Run `dig +noall +answer boisestate.edu` four more times. Then run `dig +noall +answer example.com`, wait three seconds, run it again.

TTL FOR BOISESTATE.EDU	TTL FOR EXAMPLE.COM, TWICE

One of them never moves and the other does, sometimes *upward*. Explain both. You need two facts: a TTL counts down while an answer sits in a cache, and `/etc/resolv.conf` here lists **two** resolvers.

ROUND 2, CONTINUED · THE SWITCHES THAT MATTER

One sentence each: what did this switch change about the output?

COMMAND	WHAT IT CHANGED
<code>dig +short boisestate.edu</code>	
<code>dig +noall +answer boisestate.edu</code>	
<code>dig boisestate.edu MX</code>	
<code>dig boisestate.edu NS</code>	
<code>dig boisestate.edu TXT</code>	
<code>dig boisestate.edu AAAA</code>	
<code>dig @1.1.1.1 boisestate.edu ← fails</code>	
<code>dig +trace boisestate.edu ← fails</code>	

Also try `dig +tcp boisestate.edu` (same query over TCP) and `dig +stats boisestate.edu` (timing, server used, message size). Note anything that surprised you: _____

The two that fail, and why that is the interesting part

`@1.1.1.1` times out; `+trace` stops after one step. Neither is broken software: both need to send DNS queries **directly to a server out on the internet**, and this network does not allow it. What do the two failures have in common, and what is sitting between Onyx and the internet that causes both?

The trap, and it comes back in A2

```
dig nosuchthing.boisestate.edu
dig +short nosuchthing.boisestate.edu
```

What was the **status:** field? And what exactly did the second command print?

In one sentence: why is `+short` dangerous when you are debugging, even though it is convenient when you are scripting?

ROUND 3 · STOP TYPING YOUR PASSWORD

Key based login, then connection multiplexing, then measure what multiplexing bought you.

```
ssh-keygen -t ed25519 -C "you@boisestate.edu"
ssh-copy-id <username>@onyx.boisestate.edu
# then on Onyx, because ssh ignores a loose ~/.ssh:
chmod 700 ~/.ssh ; chmod 600 ~/.ssh/authorized_keys
# then, on your laptop, with the Host block from the activity page:
ssh -O exit onyx 2>/dev/null ; time ssh onyx true ; time ssh onyx true
```

MEMBER	LAPTOP OS	COLD SSH	WARM SSH	SPEEDUP, AND WHY

Setup costs a handful of round trips. Given the cold time, roughly how many round trips did it spend, and what was it doing in them?

ROUND 4 · curl: HTTP, AND WHERE THE TIME GOES

Lines starting with > are what you sent. Lines starting with < are what the server replied.

```
curl -v http://example.com/
```

FIND THIS	WRITE IT HERE
The request line you sent	
The Host: header	
The status line that came back	
The Content-Type	

COMMAND	WHAT IT CHANGED
curl -I http://example.com/	
curl -IL http://www.boisestate.edu/	
curl -H 'User-Agent: cs425' -v http://example.com/	
curl --connect-timeout 3 -m 5 http://example.com/	

Where the time actually goes · you will use this in A2 and A3

```
curl -o /dev/null -s -w 'dns=%{time_namelookup} connect=%{time_connect} tls=%{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total} code=%{http_code}\n' https://www.boisestate.edu/
```

DNS	CONNECT	TLS	TTFB	TOTAL	CODE

1. Which phase took the longest?

2. dns is name resolution and connect is the TCP handshake. What is happening between connect and ttfb?

3. Run it against http:// instead of https://. Which number collapses to zero, and what does that tell you about what it was measuring?

ROUND 5 · nc: BE THE CLIENT YOURSELF

nc knows no protocol at all. It opens a connection and moves bytes, which means you get to be the protocol. That is exactly what P1 asks you to write.

```
nc -vz onyx.boisestate.edu 22    # something is listening
nc -vz onyx.boisestate.edu 9     # nothing is
```

PORT	WHAT NC PRINTED, WORD FOR WORD	HOW LONG
22		
9		

Those are two completely different failures. Telling them apart is most of A2.

Speak HTTP by hand

```
{ printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n'; sleep 3; } | nc example.com 80
```

Why the sleep 3? Onyx has nmap-ncat, which closes the connection the moment stdin runs out. Without the sleep, your request goes out and nc hangs up before the reply lands. Run it once without, add `-v`, and read the bytes sent versus bytes received line.

1. What was the very first line the server sent back?

2. Drop the **Host:** header and run it again. What changed, and why does HTTP/1.1 require it?

3. Change every `\r\n` to `\n`. What does the server do? **P1 is a mail client and SMTP has the same rule**, so this is worth getting wrong now rather than at 2am in week 4.

Listen, and be listened to

You are all on one machine, so one of you can be the server. Bind to **127.0.0.1**, never **0.0.0.0**, and kill it when you are done.

```
PORT=$((10000 + $(id -u) % 20000)); echo "$PORT"
ss -ltn | grep ":$PORT" || echo "free"
nc -l 127.0.0.1 "$PORT"           # blocks, waiting. Another member: nc 127.0.0.1 <PORT>
```

Whose port did you connect to, and what did you send? Confirm with `ss -ltn` that nothing of yours is still listening afterwards.

ROUND 6 · ping: THE ONE THAT IS NOT AN APPLICATION TOOL

Everything above spoke an application protocol. This one does not: ICMP rides directly in IP, and has no ports and no idea what a service is.

```
ping -c 5 onyx.boisestate.edu      # also try: -i 0.5  -s 1000  -W 2  -w 5  -n  -q
ping -c 3 example.com             # the host you fetched a page from in Round 4
```

TARGET	PACKET LOSS	DID CURL REACH IT EARLIER?
onyx.boisestate.edu		
example.com		

Those two rows contradict each other. That contradiction is the whole point of this round, and of Round 3 in A2.

1. What layer does ICMP live at, and what does that mean about ports?

2. Onyx answers ping. Does that prove the SSH server on it is running? Why not?

3. **example.com** does not answer ping, and also served you a web page. So what did the failed ping actually tell you? Name at least two different things that could produce that result.

ROUND 7 · MAKE IT STICK

In A2 you will type one curl command more than a dozen times. Teach the shell a new word instead, and make it survive a logout.

1. Find out where bash reads its configuration

On Red Hat a login shell reads ~/.bash_profile, and the default one sources ~/.bashrc. Confirm it on your own account.

```
cat ~/.bash_profile
head -20 ~/.bashrc
```

Is there a line near the top of ~/.bashrc like [-z "\$PS1"] && return? On Onyx there is not, and that is why step 4b works. On a machine that has one, the file gives up immediately for any shell that is not interactive. Write what you found.

2. Add the function, at the bottom of ~/.bashrc

```
# CS425: probe a host and port, report how long each phase took
probe() {
    curl -sS -o /dev/null -m 8 \
        -w 'connect=%{time_connect} total=%{time_total} code=%{http_code}\n' \
        "http://$1:$2/"
}
```

3. Load it and check that it took

```
source ~/.bashrc
type probe          # should say "probe is a function"
declare -f probe     # prints the definition back
probe example.com 80
```

What did probe example.com 80 print?

4. Prove it is permanent, which is the actual point

a. Log out of Onyx completely, log back in, and run probe example.com 80 without sourcing anything. Did it work?

b. From your laptop: ssh onyx probe example.com 80. No prompt, no terminal, just one command. It should work, because bash reads ~/.bashrc when a remote shell daemon starts it. In one sentence: why would this fail on a machine whose ~/.bashrc began with the guard line from step 1?

c. Why a function and not an alias? Try alias probe='curl ...' and see what happens when you need \$1 and \$2 in the middle of the command.

IF YOU FINISH EARLY

Put the same function in your laptop's shell configuration so it is there whichever machine you work from in A2: ~/.zshrc on macOS, ~/.bashrc on Linux and in Git Bash. Then write a second function probes that prints the full five phase breakdown from Round 4 instead of two numbers.

Graded pass/fail on a serious attempt at every round. Hand this in before you leave, with every name at the top.